Lec05：面向对象经典的设计原则 – 其他原则

Classic design principles of object-oriented programming - others

课程编号：22WHSE22006（2023版培养计划）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋

Email：xinguodong@163.com



QQ 群：软件构造2025

群　号：311528424

本章内容

序号	设计原则	一句话归纳	目的
1	KISS 原则/懒人原则 Keep It Simple and Stupid	尽量保持简单(讨论如何做的问题)	KISS 原则是保持代码可读和可维护的重要手段
2	YAGNI You Aren't Gonna Need It	不要做过度设计（讨论要不要做的问题)	避免复杂性：减少因预测性开发导致的代码臃肿和维护成本
3	DRY 原则 Don't Repeat Yourself	不要写重复的代码	减少代码冗余，提高代码可维护性和可读性，降低出错率和维护成本
4	迪米特法则 Law of Demeter，LoD 最少知识原则 Least Knowledge Principle，LKP	只与直接朋友通信/不要和陌生人说话。	通过限制对象间的交互范围来降低系统耦合度，提升模块独立性和可维护性
5	组合复用原则 Composite Reuse Principle，CRP	优先用对象组合，而不是继承实现代码复用	提高代码的灵活性和可维护性，减少类之间的紧密耦合

1. KISS 原则

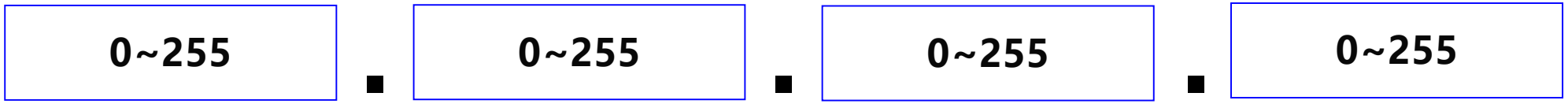
Keep **I**t **S**imple and **S**tupid，**KISS**

1.1 KISS原则的含义

- KISS 原则源于大卫·马麦特的电影理论，后来被逐渐延伸扩展到其他领域。英文表述：
  - Keep It Simple and Stupid.
  - Keep It Simple, Stupid.
  - Keep It Short and Simple.
  - Keep It Simple and Straightforward.
- 核心意思：尽量保持简单
- KISS 原则是保持代码可读和可维护的重要手段。代码足够简单，也就意味着很容易读懂，bug 比较难隐藏。即便出现 bug，修复起来也比较简单。
- 如何落地?

1.2 代码行数越少就越“简单”？

- 以下是 IP (V4) 地址校验的三种写法，哪一段代码最符合KISS 原则?




```
// 第一种实现方式：使用正则表达式
public boolean isValidIpAddressV1(String ipAddress) {
    if (StringUtils.isBlank(ipAddress)) return false;
    String regex = "^(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|[1-9])\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)$";
    return ipAddress.matches(regex);
}
```

```java
// 第二种实现方式：使用现成的工具类
public boolean isValidIpAddressV2(String ipAddress) {
    if (StringUtils.isBlank(ipAddress)) return false;
    String[] ipUnits = StringUtils.split(ipAddress, '.');
    if (ipUnits.length != 4) {
        return false;
    }
    for (int i = 0; i < 4; ++i) {
        int ipUnitIntValue;
        try {
            ipUnitIntValue = Integer.parseInt(ipUnits[i]);
        } catch (NumberFormatException e) {
            return false;
        }
        if (ipUnitIntValue < 0 || ipUnitIntValue > 255) {
            return false;
        }
        if (i == 0 && ipUnitIntValue == 0) {
            return false;
        }
    }
    return true;
}
```

```java
// 第三种实现方式：不使用任何工具类
public boolean isValidIpAddressV3(String ipAddress) {
    char[] ipChars = ipAddress.toCharArray();
    int length = ipChars.length;
    int ipUnitIntValue = -1;
    boolean isFirstUnit = true;
    int unitsCount = 0;
    for (int i = 0; i < length; ++i) {
        char c = ipChars[i];
        if (c == '.') {
            if (ipUnitIntValue < 0 || ipUnitIntValue > 255) return false;
            if (isFirstUnit && ipUnitIntValue == 0) return false;
            if (isFirstUnit) isFirstUnit = false;
            ipUnitIntValue = -1;
            unitsCount++;
            continue;
        }
        if (c < '0' || c > '9') {
            return false;
        }
        if (ipUnitIntValue == -1) ipUnitIntValue = 0;
        ipUnitIntValue = ipUnitIntValue * 10 + (c - '0');
    }
    if (ipUnitIntValue < 0 || ipUnitIntValue > 255) return false;
    if (unitsCount != 3) return false;
    return true;
}
```

1.2 代码行数越少就越“简单”？

- 第一种实现方式利用的是正则表达式，只用三行代码就把这个问题搞定。它的代码行数最少，是否就最符合 KISS 原则呢?

代码段落	分析	是否符合KISS
1	一方面，正则表达式本身复杂，写出完全没有 bug 的正则表达本身就比较有挑战；另一方面，并不是每个程序员都精通正则表达式。这种实现方式会导致代码的可读性和可维护性变差	

1.2 代码行数越少就越“简单”？

- 第一种实现方式利用的是正则表达式，只用三行代码就把这个问题搞定。它的代码行数最少，是否就最符合 KISS 原则呢?

代码段落	分析	是否符合KISS
2	第二种实现方式使用了 StringUtils 类、Integer 类提供的一些现成的工具函数，来处理 IP地址字符串。从可读性上来说，第二种实现方式的代码逻辑更清晰、更好理解	

1.2 代码行数越少就越“简单”？

- 第一种实现方式利用的是正则表达式，只用三行代码就把这个问题搞定。它的代码行数最少，是否就最符合 KISS 原则呢?

代码段落	分析	是否符合KISS
3	不使用任何工具函数，而是通过逐一处理 IP 地址中的字符，来判断是否合法。第 3 种要比第 2 种更加有难度，更容易写出 bug。	

1.2 代码行数越少就越“简单”？

- 第一种实现方式利用的是正则表达式，只用三行代码就把这个问题搞定。它的代码行数最少，是否就最符合 KISS 原则呢?

代码段落	分析	是否符合KISS
1	一方面，正则表达式本身复杂，写出完全没有 bug 的正则表达本身就比较有挑战；另一方面，并不是每个程序员都精通正则表达式。这种实现方式会导致代码的可读性和可维护性变差	
2	第二种实现方式使用了 StringUtils 类、Integer 类提供的一些现成的工具函数，来处理 IP地址字符串。从可读性上来说，第二种实现方式的代码逻辑更清晰、更好理解	
3	不使用任何工具函数，而是通过逐一处理 IP 地址中的字符，来判断是否合法。第 3 种要比第 2 种更加有难度，更容易写出 bug。	

1.3 代码逻辑复杂就违背 KISS 原则吗?

```
// KMP algorithm: a, b 分别是主串和模式串；n, m 分别是主串和模式串的长度。
public static int kmp(char[] a, int n, char[] b, int m) {
    int[] next = getNexts(b, m);
    int j = 0;
    for (int i = 0; i < n; ++i) {
        while (j > 0 && a[i] != b[j]) { // 一直找到 a[i] 和 b[j]
            j = next[j - 1] + 1;
        }
        if (a[i] == b[j]) {
            ++j;
        }
        if (j == m) { // 找到匹配模式串的了
             return i - m + 1;
        }
    }
    return -1;
}
```

1.3 代码逻辑复杂就违背 KISS 原则吗?

```
// b 表示模式串，m 表示模式串的长度
private static int[] getNexts(char[] b, int m) {
    int[] next = new int[m];
    next[0] = -1;
    int k = -1;
    for (int i = 1; i < m; ++i) {
        while (k != -1 && b[k + 1] != b[i]) {
            k = next[k];
        }
        if (b[k + 1] == b[i]) {
            ++k;
        }
        next[i] = k;
    }
    return next;
}
```

而 KMP 算法本身具有逻辑复杂、实现难度大、可读性差的特点。本身就是复杂的问题，用复杂的方法解决，并不违背 KISS 原则

1.4 如何写出满足 KISS 原则的代码

评判代码是否简单，有一个很有效的间接方法，那就是 code review。如果在 code review 的时候，同事对你的代码有很多疑问，那就说明你的代码有可能不够“简单”，需要优化

- 不要使用同事可能不懂的技术来实现代码。
- 比如前面例子中的正则表达式，还有一些编程语言中过于高级的语法等。

- 不要重复造轮子，要善于使用已经有的工具类库。
- 经验证明，自己实现类库，出 bug 的概率会更高，维护的成本也比较高

- 不要过度优化。
- 不要过度使用技巧（比如，位运算代替算术运算、使用一些过于底层的函数等）来优化代码，牺牲代码可读性

2. YAGNI 原则

You **A**ren't **G**onna **N**eed **I**t, **YAGNI**

2.1 什么是 YAGNI 原则

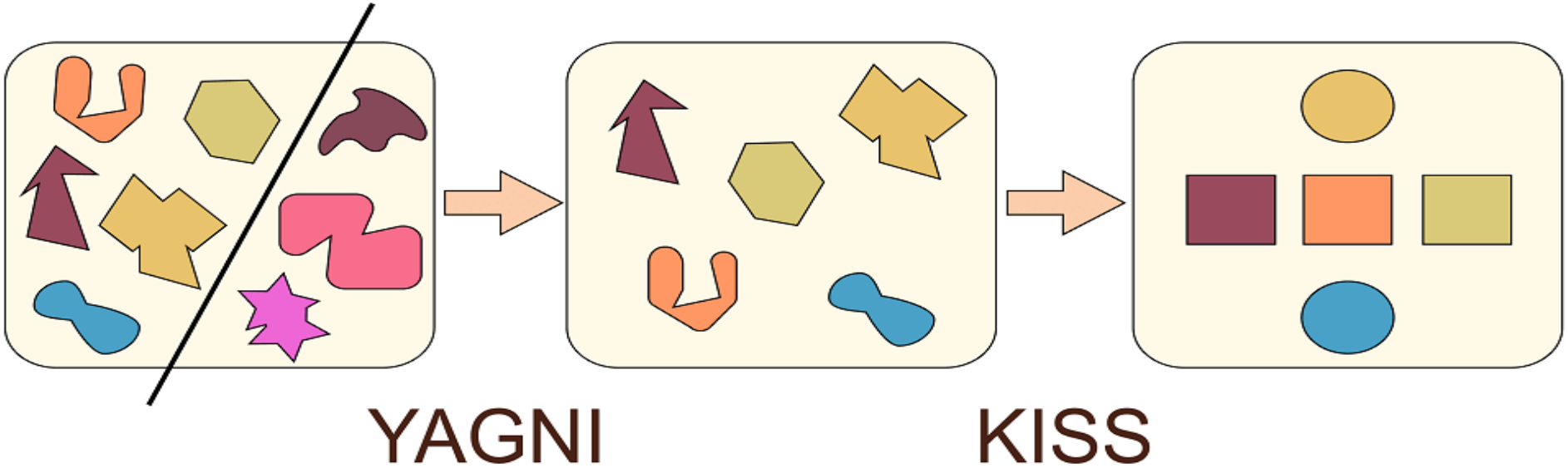
- 开发者只实现当前需要的功能，避免过度设计和提前优化。这个原则帮助团队集中精力解决实际问题，而不是预测未来可能的需求
- 这条原则的核心思想就是：不要做过度设计

2.2 遵守 YAGNI 原则是否就不必考虑代码的扩展性?

- 例如：我们的系统暂时只用 Redis 存储配置信息，以后可能会用到 ZooKeeper，根据YAGNI原则，在未用到 ZooKeeper 之前，我们没必要提前编写这部分代码。
- 当然，这并不是说我们就不需要考虑代码的扩展性。我们还是要预留好扩展点，等到需要的时候，再去实现 ZooKeeper 存储配置信息这部分代码

2.3 YAGNI 原则与 KISS 原则的对比

YAGNI 原则说的是“要不要做”的问题（当前不需要的就不要做）



KISS 原则讲的是“如何做”的问题（尽量保持简单）

3. DRY 原则

Don’t **R**epeat **Y**ourself，**DRY**

3.1 DRY 原则的理解

- DRY 原则的英文描述为：Don’t Repeat Yourself。中文直译为：不要重复自己
- 它应用在编程中，可以理解为：不要写重复的代码。
- 很多人对这条原则存在的误解。实际上，重复的代码不一定违反 DRY 原则，而且有些看似不重复的代码也有可能违反 DRY 原则。
- 三种典型的代码重复情况，它们分别是：
  - 实现逻辑重复
  - 功能语义重复
  - 代码执行重复

3.2 违反 DRY 原则典型情况 1：实现逻辑重复

```
public void sendBook() {  //上层模块
    try {
        this.service.sendBook(); // 调用service对象的sendBook()方法
    } catch (Throwable t) {
        this.notification.send(new SendFailure(t));
        throw t;
    }
}

public void sendChapter() {
    try {
        this.service.sendChapter();
    } catch (Throwable t) {
        this.notification.send(new SendFailure(t));
        throw t;
    }
}

public void startTranslation() {
    try {
        this.service.startTranslation();
    } catch (Throwable t) {
        this.notification.send(new SendFailure(t));
        throw t;
    }
}
```



代码结构有重复：

三个方法调用的业务代码不同，但它们的结构是一致的，其基本流程可以理解为：

- ①调用业务函数；
- ②如果出错，发通知。

3.2 违反 DRY 原则典型情况 1：实现逻辑重复-解决方案

```
public class TranslateController { // 重构
    private void executeTask(final Runnable runnable) {
        try {
            runnable.run();
        } catch (Throwable t) {
            this.notification.send(new SendFailure(t));
            throw t;
        }
    }

    public void sendBook() {
        executeTask(this.service::sendBook);
    }

    public void sendChapter() {
        executeTask(this.service::sendChapter);
    }

    public void startTranslation() {
        executeTask(this.service::startTranslation);
    }
}
```

解决思路：

结构上的重复①调用业务函数；②如果出错，发通知。

将结构提取出来，封装成私有方法

使用 Java 中的方法引用，简化代码书写（Lambda->Method Reference）

再有一些通用的结构调整，比如在任务执行前后加一些日志信息，这样的改动就可以放到 executeTask 这个函数里，而不用四处去改写

3.2 违反 DRY 原则典型情况 2：功能语义重复

```
public boolean isValidIp(String ipAddress) {
    if (StringUtils.isBlank(ipAddress)) return false;
    String regex = "^(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|[1-9])\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)$";
    return ipAddress.matches(regex);
}

public boolean checkIfIpValid(String ipAddress) {
    if (StringUtils.isBlank(ipAddress)) return false;
    String[] ipUnits = StringUtils.split(ipAddress, '.');
    if (ipUnits.length != 4) {
        return false;
    }
    for (int i = 0; i < 4; ++i) {
       //代码省略
    }
    return true;
}
```

两个函数的命名不同，实现逻辑不同，但功能是相同的，都是用来判定 IP 地址是否合法。解决方案是删除一个方法即可

3.2 违反 DRY 原则典型情况 2：功能语义重复-解决方案

```
public boolean isValidIp(String ipAddress) {
    if (StringUtils.isBlank(ipAddress)) return false;
    String regex = "^(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|[1-9])\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)\\."
            + "(1\\d{2}|2[0-4]\\d|25[0-5]|[1-9]\\d|\\d)$";
    return ipAddress.matches(regex);
}
// 删除重复的接口。
```



3.2 违反 DRY 原则典型情况 3：代码执行重复

```
public class UserService {
    private UserRepo userRepo; //通过依赖注入或者 IOC 框架注入
    public User login(String email, String password) { //校验用户登录是否成功
        boolean existed = userRepo.checkIfUserExisted(email, password);

        if (!existed) {   // ... throw AuthenticationFailureException...   }
        User user = userRepo.getUserByEmail(email);
        return user;
    }
}

public class UserRepo {
    public boolean checkIfUserExisted(String email, String password) {
        if (!EmailValidation.validate(email)) {  // ... throw InvalidEmailException...  }
        if (!PasswordValidation.validate(password)) { // ... throw InvalidPasswordException... }
        //...query db to check if email&password exists...
    }
    public User getUserByEmail(String email) {
        if (!EmailValidation.validate(email)) {  // ... throw InvalidEmailException... }
        //...query db to get user by email...
    }
}
```

问题①：email 的校验逻辑被执行了两次。将校验逻辑放到 UserService 即可解决该问题

3.2 违反 DRY 原则典型情况 3：代码执行重复

```
public class UserService {
    private UserRepo userRepo; //通过依赖注入或者 IOC 框架注入
    public User login(String email, String password) { //校验用户登录是否成功
        boolean existed = userRepo.checkIfUserExisted(email, password);

        if (!existed) {   // ... throw AuthenticationFailureException...   }
        User user = userRepo.getUserByEmail(email);
        return user;
    }
}

public class UserRepo {
    public boolean checkIfUserExisted(String email, String password) {
        if (!EmailValidation.validate(email)) {  // ... throw InvalidEmailException...  }
        if (!PasswordValidation.validate(password)) { // ... throw InvalidPasswordException... }
        //...query db to check if email&password exists...
    }
    public User getUserByEmail(String email) {
        if (!EmailValidation.validate(email)) {  // ... throw InvalidEmailException... }
        //...query db to get user by email...
    }
}
```

问题②：login() 函数并不需要调用 checkIfUserExisted() 函数，只需调用 getUserByEmail() 函数，从数据库中获取到用户的 email、password 等信息，然后跟用户输入的 email、password 信息做对比，依次判断是否登录成功即可。

3.2 违反 DRY 原则典型情况 3：代码执行重复-解决方案

```
public class UserService {
    private UserRepo userRepo; // 通过依赖注入或者 IOC 框架注入
    public User login(String email, String password) {
        if (!EmailValidation.validate(email)) { // ... throw InvalidEmailException...  }
        if (!PasswordValidation.validate(password)) { // throw InvalidPasswordException  }
        User user = userRepo.getUserByEmail(email);
        if (user == null || !password.equals(user.getPassword())) {
            // ... throw AuthenticationFailureException...
        }
        return user;
    }
}
```

问题②的解决：删除了 checkIfUserExisted()

问题①的解决：email 和 password 校验放到此处，避免重复

此处的优化是很有必要的：因为数据库这类的 I/O 操作比较耗时。应尽量减少 I/O 操作

```
public class UserRepo {
    public class UserRepo {
        public boolean checkIfUserExisted(String email, String password) {
            //...query db to check if email&password exists
        }
        public User getUserByEmail(String email) { //...query db to get user by email... }
    }
}
```

3.3 练习：读如下代码，使其满足 DRY

- 如下代码的业务逻辑：章节只有在审核通过之后，才能去做后续的处理，比如，章节的翻译。所以，这里的 editChapter 方法最后那个参数表示是否审核通过。

```
if (user.isEditor()) {
    service.editChapter(chapterId, title, content, true);
} else {
    service.editChapter(chapterId, title, content, false);
}
```

- if 和 else 两段代码除了最后一个参数不同之外，几乎一模一样的。这也是一种重复代码。
- 如何重构?

3.3 练习：解决方案

```
if (user.isEditor()) {
    service.editChapter(chapterId, title, content, true);
} else {
    service.editChapter(chapterId, title, content, false);
}
```

把 user.isEditor() 的结果赋值给 approved 变量，而不是直接作为参数传给 editChapter

```
boolean approved = user.isEditor();
service.editChapter(chapterId, title, content, approved);
```



甚至可以提取一个函数 isApproved 用来封装 `user.isEditor()` 。这样，就把变化都放到这个函数里

```
boolean approved = isApproved(user);
service.editChapter(chapterId,title,content,approved);

private boolean isApproved(final User user) {
    return user.isEditor();
}
```



4. 迪米特法则

Law **o**f **D**emeter，LoD

4.1 迪米特法则是实现“高内聚低耦合”的重要手段

“高内聚、低耦合”是一个非常重要的设计思想，能够有效地提高代码的可读性和可维护性，缩小功能改动导致的代码改动范围。

- 指导不同粒度代码的设计与开发，比如系统、模块、类，甚至是函数，
- 可以应用到不同的开发场景中，比如微服务、框架、组件、类库等。

什么是高内聚：

- 相近的功能应该放到同一个类中
- 相近的功能往往会被同时修改，放到同一个类中，修改会比较集中，代码容易维护
- 单一职责原则是实现代码高内聚非常有效的设计原则

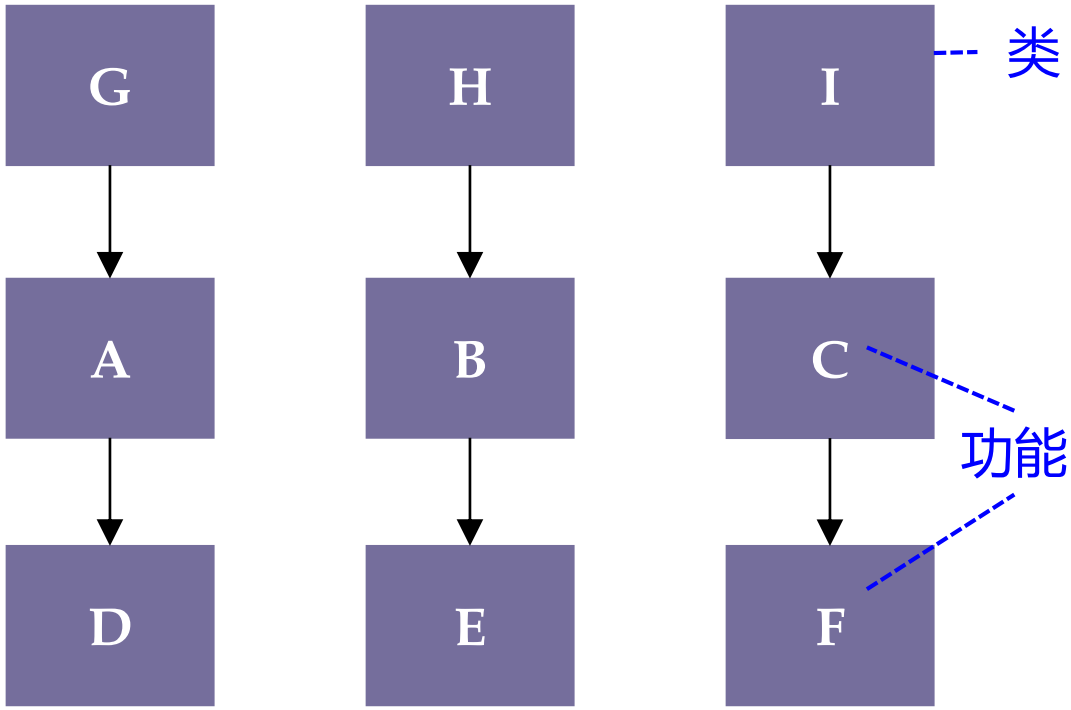
什么是松耦合：

- 松耦合体现在类与类之间的依赖关系简单清晰。
- 即便两个类有依赖关系，一个类的代码改动，不会或者很少导致依赖类的代码改动。
- 依赖注入、接口隔离、基于接口而非实现编程，迪米特法则，都是为了实现代码的松耦合。

4.1 迪米特法则是实现“高内聚低耦合”的重要手段

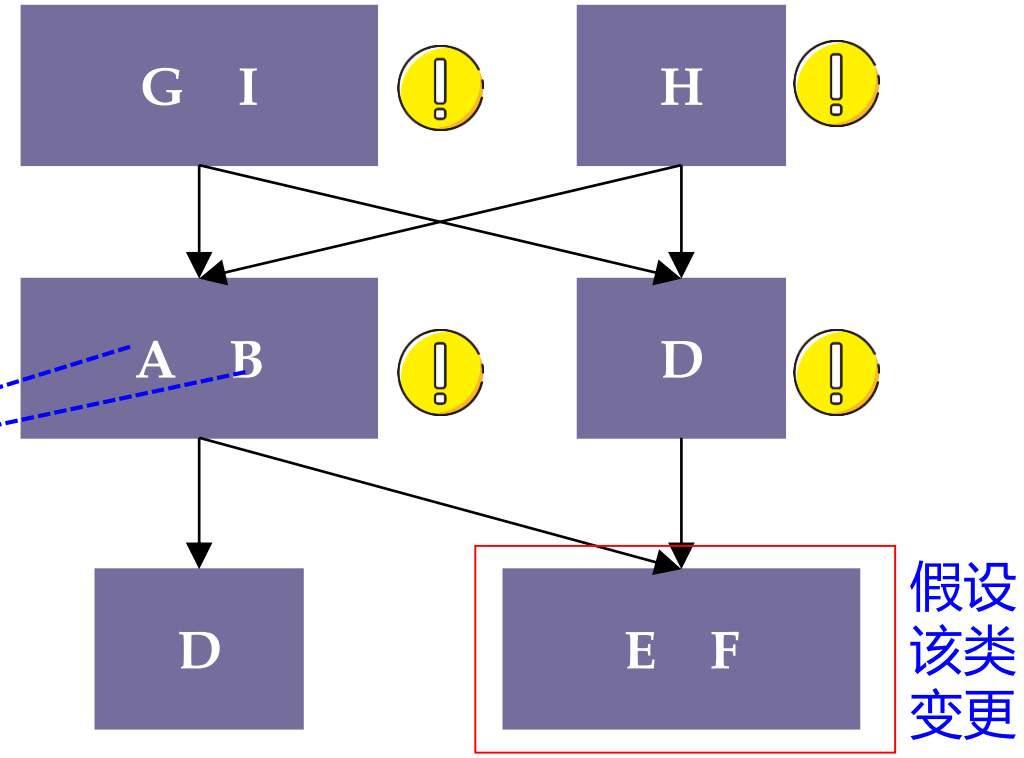
高内聚、松耦合

注意：每个方块代表一个类，类中字母表示功能



- 类的粒度比较小，职责单一。相近的功能都放到了一个类中，不相近的功能被分割到了多个类中，类更加独立，代码的内聚性更好。
- 因为职责单一，所以每个类依赖的类就会比较少，代码低耦合。一个类的修改，只会影响到一个依赖类的代码改动

低内聚、紧耦合



- 类粒度比较大，低内聚，功能大而全，不相近的功能放到了一个类中，导致很多其他类都依赖这个类。当我们修改这个类的某一个功能代码的时候，会影响依赖它的多个类
- 例如 EF 功能组成的类：其代码变更会影响依赖它的多个类。这就是所谓的“牵一发而动全身”。

4.2 迪米特法则的描述

- 迪米特法则来自于 1987 年美国东北大学 (Northeastern University) 一个名为“Demeter”的研究项目。又称为最少知识原则（Least Knowledge Principle，LKP)，描述如下：

Each unit should have only limited knowledge about other units: only units “closely” related to the current unit. Or: Each unit should only talk to its friends; Don’t talk to strangers.

每个模块只应该了解那些与它关系密切的模块的有限知识。或者说，每个模块只和自己的朋友“说话”，不和陌生人“说话”

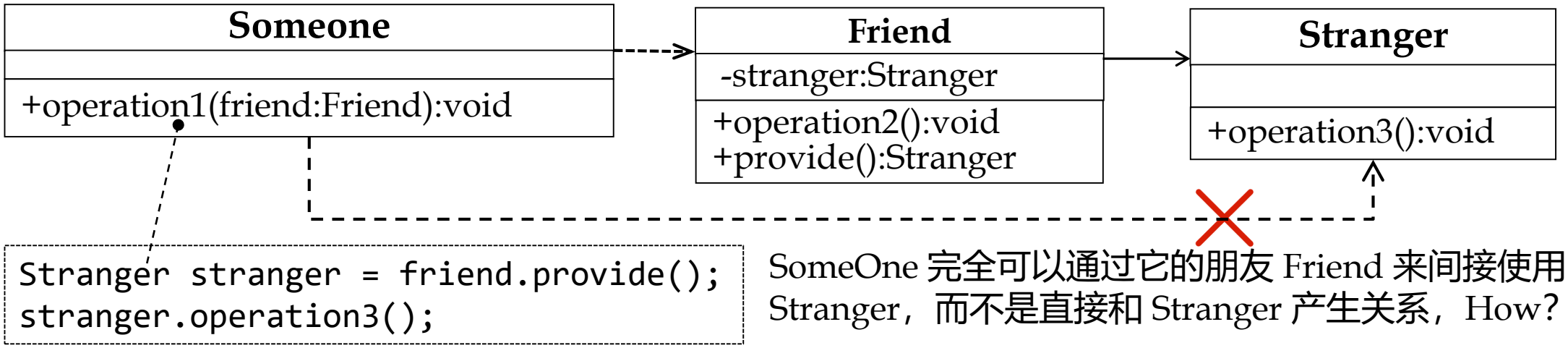
理解为

以类为例：不该有直接依赖关系的类之间，不要有依赖；有依赖关系的类之间，尽量只依赖必要的接口（也就是定义中的“有限知识”）

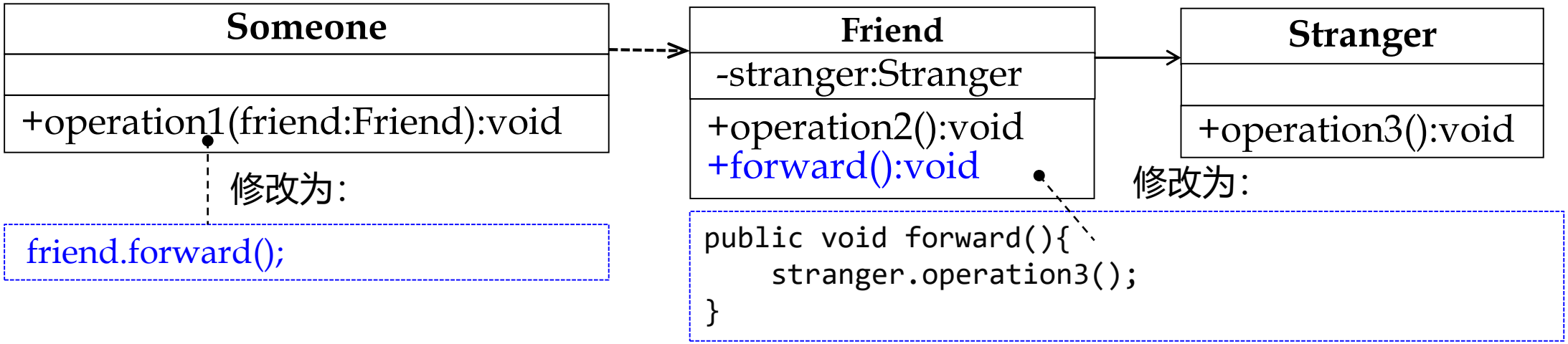
① 不该有直接依赖关系的类之间，不要有依赖

② 有依赖关系的类之间，尽量只依赖必要的接口

4.3 不该有直接依赖关系的类之间，不要有依赖



解决方案：



4.4 有依赖关系的类之间，尽量只依赖必要的接口

- 在讲单一职责，讨论“类的职责是否设计得越单一越好？”时用序列化/反序列化的例子：

```
public class Serialization {
    public String serialize(Object object) {
        String serializedResult = ...;
        //...
        return serializedResult;
    }
    public Object deserialize(String str) {
        Object deserializedResult = ...;
        //...
        return deserializedResult;
    }
}
```

见：面向对象经典的设计原则-solid.pptx，P6-7

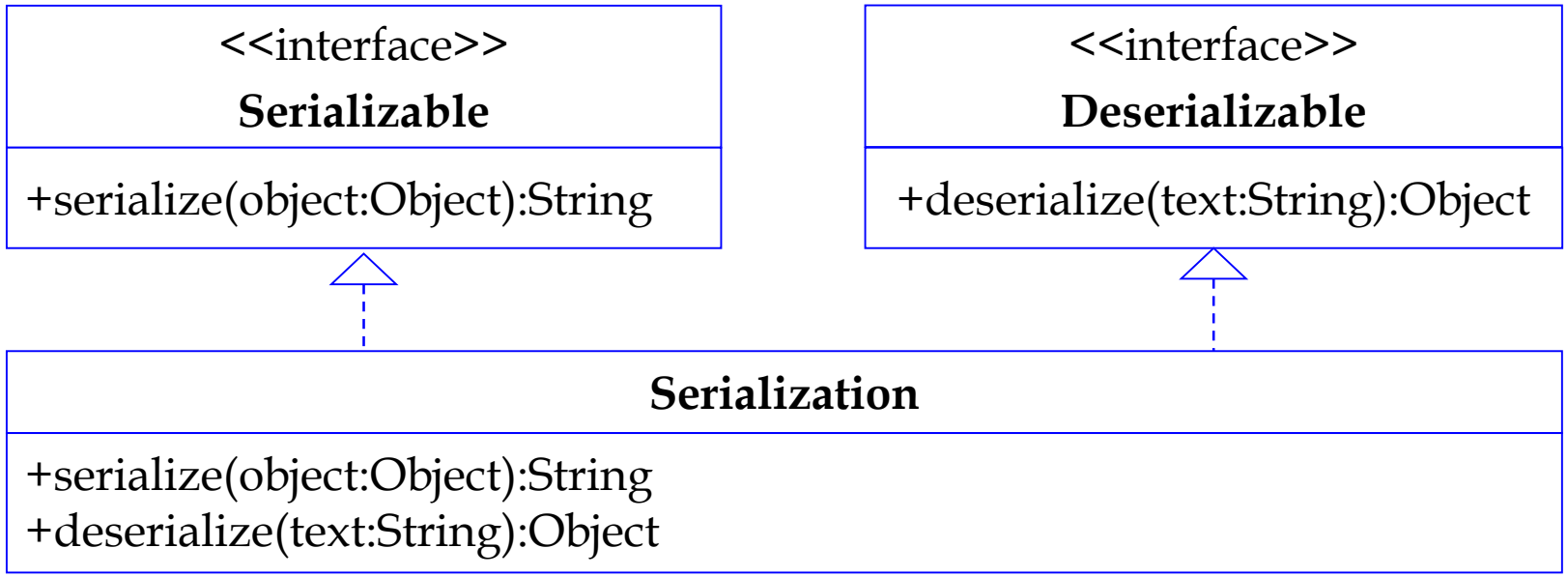
```
public class serializer {
    public String serialize(Object object) {
        String serializedResult = ...;
        //...
        return serializedResult;
    }
}
```

```
public class Deserializer {
    public Object deserialize(String str) {
        Object deserializedResult = ...;
        //...
        return deserializedResult;
    }
}
```

我们认为，将之拆成两个类 Serializer 类和 Deserializer 类后，尽管职责单一，但：如果我们修改了序列化的实现方式，比如从JSON 换成了 XML，那反序列化的实现逻辑也需要一并修改。即：代码的可维护性变差；代码的内聚性显然没有拆分之前高。

4.4 有依赖关系的类之间，尽量只依赖必要的接口

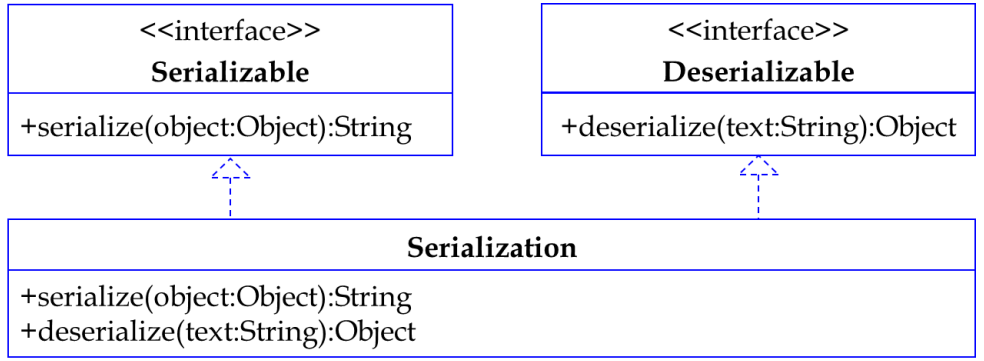
- 假设在我们的项目中，有些类只用到了序列化操作，而另一些类只用到反序列化操作，拆分就变成合理的了。
- 既不想违背高内聚的设计思想，也不想违背迪米特法则，那我们该如何避免拆分后的问题？
- 实现思路：



4.4 有依赖关系的类之间，尽量只依赖必要的接口

```java
public interface Deserializable {
    Object deserialize(String text);
}
```

```java
public interface Serializable {
    String serialize(Object object);
}
```



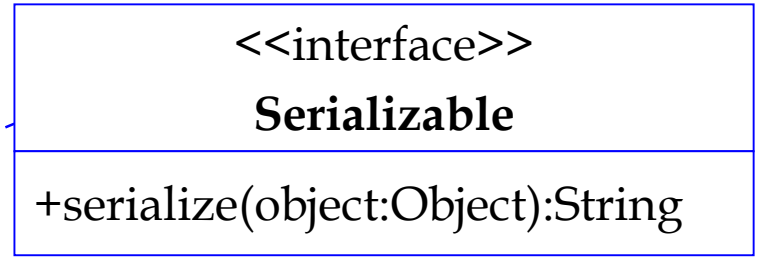

```java
public class Serialization implements Serializable, Deserializable {
    @Override
    public String serialize(Object object) {
        String serializedResult = ...;
        // ...
        return serializedResult;
    }
    @Override
    public Object deserialize(String str) {
        Object deserializedResult = ...;
        //...
        return deserializedResult;
    }
}
```

4.4 有依赖关系的类之间，尽量只依赖必要的接口

```
public class DemoClass_1 {

    private Serializable serializer; //有限感知

    public DemoClass_1(Serializable serializer) {
        this.serializer = serializer;
    }
    // ....
}
```

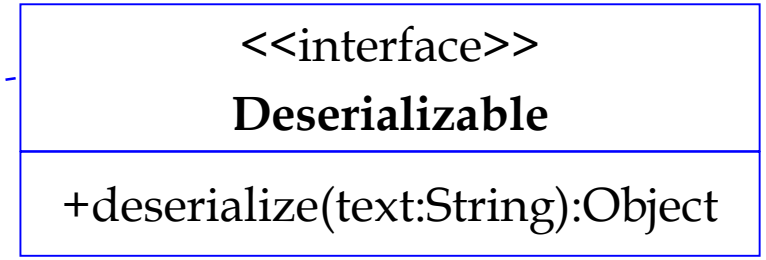


DemoClass_1 用构造方法注入序列化对象，只能用序列化功能。对反序列化操作无感知：依赖有限接口

```
public class DemoClass_2 {

    private Deserializable deserializer; //有限感知

    public DemoClass_2(Deserializable deserializer) {
        this.deserializer = deserializer;
    }
    //...
}
```



DemoClass_2 用构造方法注入Deserializable 对象，只能用反序列化功能。对序列化操作无感知

4.5 练习：重构如下代码

String name = book.getAuthor().getName();

↓

可以从封装的角度看待代码。也可以从迪米特法则的角度看（不要和陌生人说话）。

↓

想写出上面这段代码，是不是必须得先了解 Book 和 Author 这两个类的实现细节？也就是说，我们必须得知道，作者的姓名是存储在作品的作者字段里的。

↓

这时你就要注意了：当你必须得先了解一个类的细节，才能写出代码时，这只能说明一件事，这个封装是失败的。

↓

Martin Fowler给这种坏味道起的名字叫过长的消息链，而有人则给它起了一个更为夸张的名字：火车残骸（Train Wreck），形容这样的代码像火车残骸一般，断得一节一节的

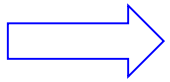
4.5 练习：解决方案

- 解决这种代码的重构手法叫隐藏委托关系（Hide Delegate），说得更直白一些就是，把这种调用封装起来

```
public class Book {
    //...
    public String getAuthorName() {
        return this.author.getName();  // 将作者的名字在book中隐藏
    }
    //...
}
```

- 代码的调用变成了如下，隐藏委托关系

```
String name = book.getAuthor().getName();
```



```
String name = book.getAuthorName();
```

火车残骸这种坏味道的产生是缺乏对于封装的理解，因为封装这件事并不是很多低层次程序员编码习惯的一部分，他们对封装的理解停留在数据结构加算法的层面上

5. 组合复用原则

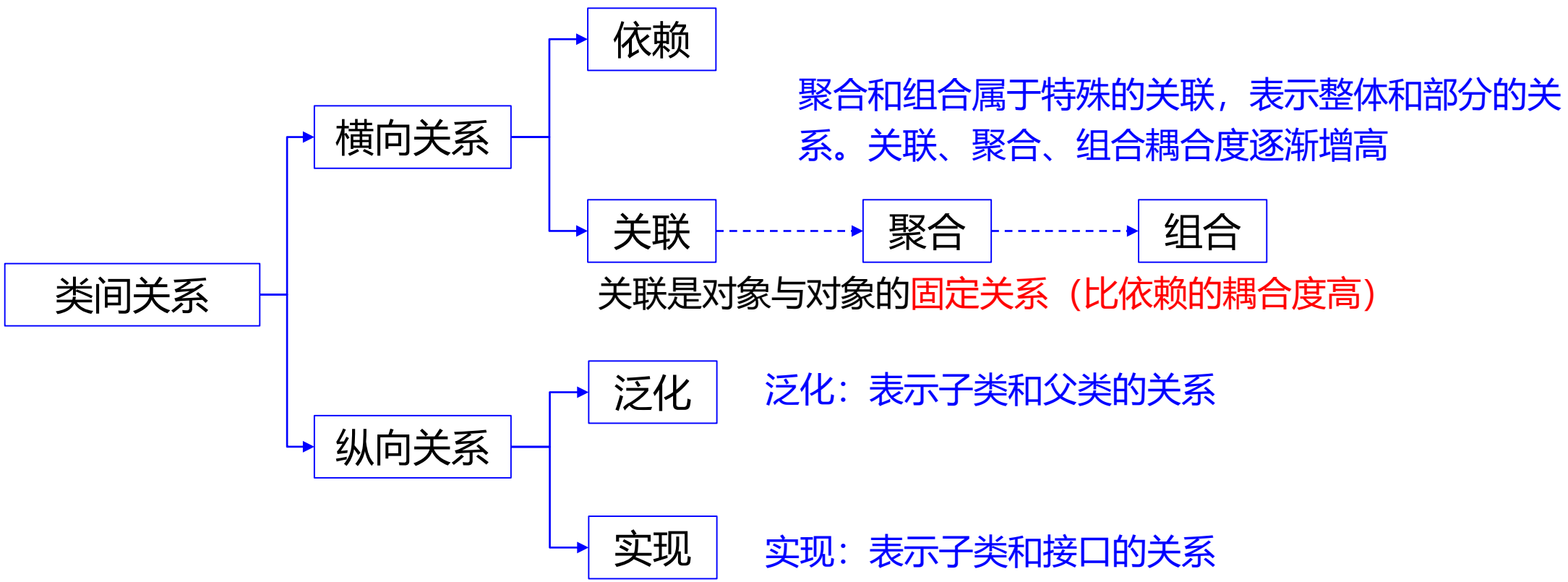
Composite Reuse Principle，CRP

5.1 组合复用原则的核心思想

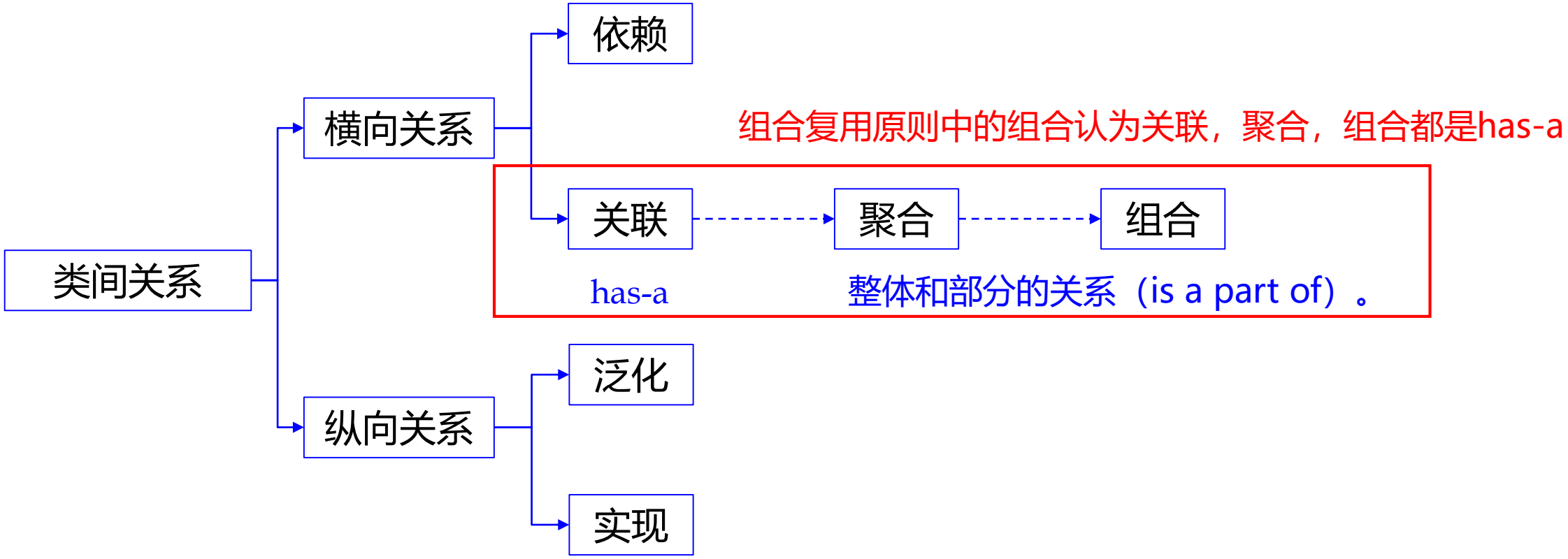
- 组合复用原则（Composite Reuse Principle, CRP），也被称为合成/聚合复用原则。
- 核心思想是：优先使用对象组合/聚合，而不是通过继承来实现代码复用。
- 该原则旨在提高代码的灵活性和可维护性，减少类之间的紧密耦合

5.2 组合复用原则的理解

组合复用的核心理念还是降耦

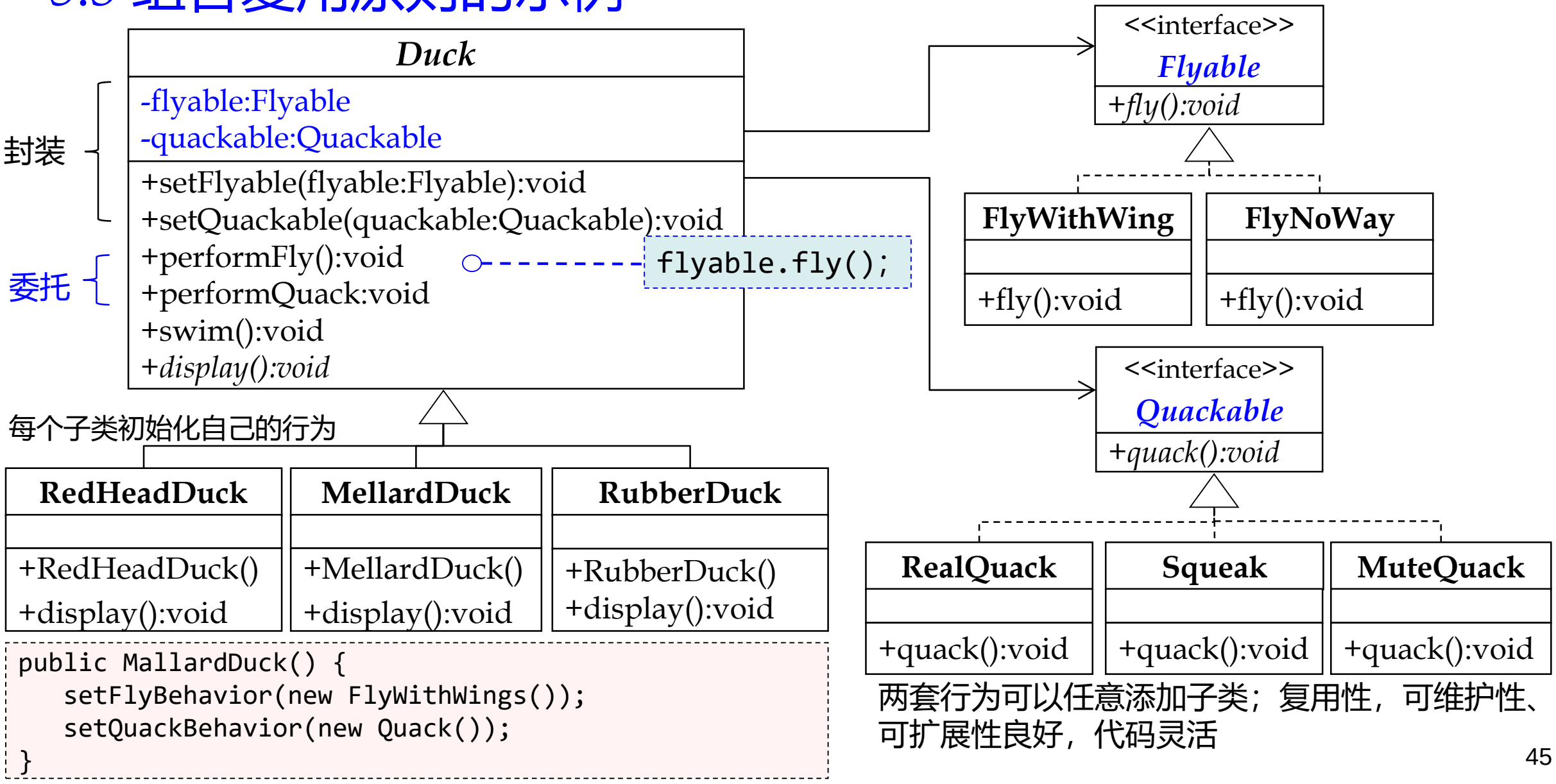


5.2 组合复用原则的理解



耦合强度：横向关系耦合度较低，纵向关系耦合度较高；
依赖<关联<聚合<组合<泛化（继承）

5.3 组合复用原则的示例




```
public MallardDuck() {
   setFlyBehavior(new FlyWithWings());
   setQuackBehavior(new Quack());
}
```

两套行为可以任意添加子类；复用性，可维护性、可扩展性良好，代码灵活

本章总结

序号	设计原则	一句话归纳	目的
1	KISS 原则/懒人原则 Keep It Simple and Stupid	尽量保持简单(讨论如何做的问题)	KISS 原则是保持代码可读和可维护的重要手段
2	YAGNI You Aren't Gonna Need It	不要做过度设计（讨论要不要做的问题)	避免复杂性：减少因预测性开发导致的代码臃肿和维护成本
3	DRY 原则 Don’t Repeat Yourself	不要写重复的代码	减少代码冗余，提高代码的可维护性和可读性，降低出错率和维护成本
4	迪米特法则 Law of Demeter，LoD 最少知识原则 Least Knowledge Principle	只与直接朋友通信/不要和陌生人说话	通过限制对象间的交互范围来降低系统耦合度，提升模块独立性和可维护性
5	组合复用原则 Composite Reuse Principle，CRP	优先使用对象组合/聚合，而不是通过继承来实现代码复用	提高代码的灵活性和可维护性，减少类之间的紧密耦合